

смотри, я хочу сделать веб приложение типа фигмы. Пользователь регистрируется и видит на экране холст с которым работает каждый раз как заходит в приложение. у приложения есть вот такой приколы: если пользователь открывает параллельно на 2 страницы приложения, то если на одной странице он что-то двигает, на второй тоже происходит движение, раз в 50 мс. Приложение использует Blazor Server и postgres. Используется не html canvas а SVG. Теперь в чем вопрос: какие есть варианты модели данных чтобы сделать сразу 4 типа фигуры (линия, круг, треугольник,
Развернуть ▾

Обработка заняла 1m 58s >

Я бы разделил задачу на геометрию фигуры, её положение на холсте, стиль и синхронизацию изменений. Тогда модель не будет привязана напрямую к SVG-разметке.

Главная рекомендация

Для такого приложения я бы начал с:

- таблицы Documents ;
- одной таблицы Elements ;
- общего положения/трансформации в обычных колонках;
- специфичной геометрии фигуры в jsonb ;
- стиля в отдельном jsonb ;
- версии элемента для синхронизации;
- мягкого удаления через DeletedAt .

Пример:

```
create table documents
(
    id uuid primary key,
    owner_id uuid not null,
    revision bigint not null default 0,
    created_at timestamptz not null default now(),
```

```

        updated_at timestamptz not null default now()
    );

create table elements
(
    id uuid primary key,
    document_id uuid not null
        references documents(id)
        on delete cascade,

    element_type text not null,

    -- Положение фигуры на холсте
    x double precision not null default 0,
    y double precision not null default 0,

    -- На будущее
    rotation double precision not null default 0,
    scale_x double precision not null default 1,
    scale_y double precision not null default 1,

    geometry jsonb not null,
    style jsonb not null default '{} '::jsonb,

    z_index integer not null default 0,

    version bigint not null default 1,
    deleted_at timestamptz null,

    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),

    check (
        element_type in (
            'line',
            'circle',
            'triangle',
            'rectangle'
        )
    )
);

```

SQL

Примеры geometry :

JSON

JSON

JSON

JSON

JSON

3/26

```
<g transform="translate(250 120) rotate(0) scale(1 1)">
```

HTML

Это очень полезное разделение:

- `geometry` описывает форму;
- `x`, `y`, `rotation`, `scale` описывают положение;
- `style` описывает внешний вид.

При перемещении фигуры меняются только `x` и `y`. Не нужно менять SVG-строку или пересчитывать все точки.

Вариант 1: сохранять весь SVG строкой

Например:

```
documents
```

```
-----
```

```
id
```

SQL

B `svg_content` :

SVG



Преимущества

- максимально просто сохранить и восстановить картинку;
- результат уже готов для отображения;
- легко сделать экспорт SVG;
- схема БД практически не меняется при добавлении новых фигур.

Недостатки

1. Невозможно нормально обновлять отдельную фигуру

Чтобы передвинуть один прямоугольник, придётся:

1. найти его внутри XML;
2. поменять атрибуты;
3. сохранить весь SVG снова.

Либо хранить и редактировать SVG через DOM, но в БД всё равно будет одна большая строка.

2. Конфликты между вкладками

Допустим:

- вкладка А передвинула круг;
- вкладка В одновременно передвинула прямоугольник;
- обе вкладки отправляют полный SVG.

Последний запрос может затереть изменение другой вкладки.

3. Сложно реализовать историю

Невозможно удобно сказать:

элемент 123 перемещён с (10, 20) на (30, 40)

Ты видишь только изменение большой XML-строки.

4. Сложно делать поиск и валидацию

Например, трудно получить:

- количество кругов;
- все элементы определённого цвета;
- фигуру с конкретным ID;
- все элементы внутри заданной области.

5. Безопасность

SVG — это не просто безобидная картинка. В него могут попадать обработчики событий, внешние ссылки и другие нежелательные конструкции. Если SVG всегда генерирует только твой сервер, риск меньше. Но импортированный или пользовательский SVG обязательно нужно очищать.

Мой вывод

Не использовать весь SVG как основную модель данных.

Но его вполне можно сохранять как:

- экспорт;

- кэш;
- snapshot документа;
- резервную копию;
- предварительно сгенерированное изображение.

Например:

```
documents
-----
id
```

SQL

Основные данные лежат в `elements`, а `svg_snapshot` иногда пересоздаётся.

Вариант 2: SVG-строка для каждого элемента

Например:

```
elements
-----
id
```

SQL

Значение:

```
<circle cx="100" cy="100" r="40"
```

XML

Это лучше, чем один SVG на весь документ, потому что можно отдельно обновлять фигуры.

Но всё равно возникают проблемы:

- геометрия остаётся непрозрачной строкой;
- для изменения нужно разбирать XML;
- сложно валидировать;
- разные фигуры имеют разные атрибуты;

- сложнее реализовать resize;
- сложно эффективно определять bounding box.

Это допустимый вариант для простого SVG-редактора, где элемент почти никогда не редактируется внутренне. Для приложения типа Figma я бы его не выбирал.

Вариант 3: отдельная таблица для каждого типа фигуры

Например:

```
lines
-----
id
document_id
x1
y1
```

SQL

```
circles
-----
id
document_id
center_x
```

SQL

```
rectangles
-----
id
document_id
x
y
```

SQL

```

triangles
-----
id
document_id
x1
y1
x2

```

SQL

y ↗

Преимущества

- строгая структура;
- никаких лишних `NULL` ;
- PostgreSQL понимает типы всех значений;
- можно делать хорошие `снесск` ;
- легко писать запросы по конкретному типу фигуры.

Главный недостаток

Все базовые операции должны работать сразу с четырьмя таблицами:

- загрузить весь холст;
- найти элемент по ID;
- удалить элемент;
- изменить `z-index` ;
- изменить стиль;
- получить все элементы пользователя;
- скопировать элемент;
- заблокировать элемент;
- создать историю изменений.

Чтобы загрузить документ, потребуется что-то вроде:

```

select ... from lines
union all
select ... from circles
union all
select ... from rectangles

```

SQL

Или четыре отдельных запроса.

При добавлении текста, изображения, стрелки, многоугольника и группы количество таблиц начнёт быстро расти.

Когда этот вариант подходит

Он подходит, если разные типы объектов действительно имеют очень разную бизнес-логику.

Например, если у тебя на холсте будут:

- диаграмма;
- видео;
- таблица;
- 3D-модель;
- форма ввода.

Но для четырёх простых SVG-фигур отдельные полностью независимые таблицы, скорее всего, будут лишними.

Более правильная версия отдельных таблиц

Если хочется строгую реляционную структуру без `jsonb`, лучше сделать **общую таблицу элементов** и дочерние таблицы геометрии.

```
elements
-----
id
document_id
element_type
x
y
rotation
z_index
style
```

SQL

```
line_geometry
-----
element_id
x1
y1
```

SQL

```
circle_geometry
```

```
-----
```

SQL

```
rectangle_geometry
```

```
-----
```

```
element_id
```

SQL

```
triangle_geometry
```

```
-----
```

```
element_id
```

```
x1
```

```
y1
```

```
x2
```

```
y2
```

SQL

Здесь `element_id` одновременно является:

- primary key;
- foreign key на `elements.id`.

Это называется примерно моделью наследования через отдельные таблицы.

Общие операции работают с `elements` :

SQL

или:

SQL

```
where id = ...;
```

А специфичная геометрия находится в дочерней таблице.

Это хороший архитектурный вариант, но он сложнее jsonb .

Вариант 4: одна широкая таблица с множеством NULL

Например:

```
elements
```

```
-----
```

```
id
```

```
type
```

```
x1
```

```
y1
```

```
x2
```

```
y2
```

```
x3
```

```
y3
```

```
center_x
```

```
center_y
```

```
radius
```

SQL

Для круга будут NULL в x1 , x2 , width и так далее. Для линии будут NULL в radius , x3 , y3 .

Я бы не рекомендовал этот вариант.

Сначала он выглядит простым, но затем появятся:

- rx ;
- ry ;
- количество сторон;
- начало и конец дуги;
- параметры стрелки;
- текст;

- размер шрифта;
- URL изображения;
- параметры градиента.

Таблица превратится в набор колонок, большая часть которых неприменима к большинству строк.

Вариант 5: название фигуры и две точки bounding box

Твоя идея примерно такая:

```
elements
-----
id
document_id
shape_type
x1
y1
```

SQL

Это может работать, но только для очень ограниченных фигур.

Прямоугольник

Для прямоугольника две противоположные точки достаточны, пока он не вращается:

```
left = min(x1, x2)
top = min(y1, y2)
width = abs(x2 - x1)
height = abs(y2 - y1)
```

Круг

Для круга две точки могут задавать bounding box.

Но тогда нужно решить:

- круг всегда вписан в квадрат;
- или это на самом деле эллипс;
- что делать, если ширина и высота bounding box отличаются.

Линия

Для линии две точки лучше считать концами линии, а не углами bounding box.

Потому что bounding box не говорит, по какой диагонали проходит линия:

\ или /

Один и тот же bounding box может соответствовать двум разным линиям.

Треугольник

Bounding box вообще не определяет треугольник однозначно.

Внутри одного прямоугольника могут быть:



А также произвольный треугольник с тремя независимо перемещаемыми вершинами.

Когда эта модель всё-таки подходит

Она работает, если фигуры являются строго фиксированными шаблонами:

- прямоугольник всегда обычный;
- круг всегда вписан в bounding box;
- треугольник всегда равнобедренный и направлен вверх;
- линия всегда идёт из левого верхнего угла в правый нижний.

Тогда:

type + bounding box

действительно определяет фигуру.

Но как только появятся:

- поворот;
- изменение отдельных вершин;
- разные виды треугольников;
- линия в другую сторону;
- стрелки;
- скруглённые прямоугольники;

этого станет недостаточно.

Поэтому bounding box лучше использовать как **вспомогательные данные**, а не как основную геометрию.

Например:

```
bbox_left  
bbox_top
```

SQL

Их можно хранить для быстрого:

- выделения рамкой;
- поиска элементов в области;
- hit testing;
- определения пересечений.

Но настоящая геометрия должна храниться отдельно.

Вариант 6: хранить SVG path d

Можно привести все фигуры к одному SVG-типу:

HTML

В таблице:

```
elements  
-----  
id  
document_id  
element_type  
path_data  
x
```

SQL

Прямоугольник, круг, линия и треугольник могут быть представлены через path .

Плюсы

- все фигуры рендерятся одинаково;
- удобно добавлять рисование пером;
- удобно для произвольных контуров;
- легко экспортировать в SVG;
- меньше типов компонентов.

Минусы

- сложно изменить радиус круга;
- сложно понять, что фигура ещё является кругом;
- `resize` становится более сложным;
- нужно парсить `path`-команды;
- семантическая информация теряется.

Хороший компромисс:

```
element_type = 'circle'
```

SQL

`path_cache` можно пересоздать, а основной источник истины остаётся структурированным.

Вариант 7: PostgreSQL `point`, `box`, `polygon`, `path`

PostgreSQL имеет собственные геометрические типы:

- `point` ;
- `line` ;
- `box` ;
- `path` ;
- `polygon` ;
- `circle` .

Теоретически можно использовать их.

Однако для SVG-редактора они редко дают большое преимущество:

- не описывают SVG-стили;
- не решают проблему разных типов элементов;
- преобразования и повороты всё равно придётся реализовывать;
- сериализация в C# может быть менее удобной;

- SVG-геометрия и геометрические типы PostgreSQL не полностью совпадают.

PostGIS для четырёх фигур тоже будет избыточным. Он полезнее для географических объектов и серьёзных пространственных запросов.

Вариант 8: Event Sourcing — хранить не фигуры, а операции

Можно сохранять действия:

```
element_operations
-----
id
document_id
element_id
operation_type
payload
```

SQL

Примеры операций:

```
{
  "operation": "create",
  "elementId": "...",
  "type": "circle",
  "geometry": {
    "radius": 50
  }
}
```

JSON

```
{
  "operation": "move",
  "elementId": "...",
  "x": 250,
}
```

JSON


```
{  
  "operation": "delete"
```

JSON

Плюсы

- история изменений;
- undo/redo;
- восстановление состояния;
- аудит;
- синхронизация между клиентами;
- можно воспроизводить документ по операциям.

Минусы

- сложнее загрузка;
- операций становится очень много;
- нужны snapshots;
- труднее исправлять ошибки;
- для первого варианта приложения это может быть слишком сложно.

На практике хороший вариант — совмещать:

```
elements = текущее состояние  
element_operations = история значимых действий
```

Не нужно обязательно записывать каждое движение мыши. Можно сохранять:

```
MoveStarted  
MoveCompleted
```

или только конечное положение.

Что я рекомендую конкретно тебе

Основная таблица элементов

```
public enum ElementType  
{  
    Line,  
    Circle,  
    Triangle,  
}
```

```

    Rectangle
}

public sealed class CanvasElement
{
    public Guid Id { get; set; }

    public Guid DocumentId { get; set; }

    public ElementType Type { get; set; }

    public double X { get; set; }

    public double Y { get; set; }

    public double Rotation { get; set; }

    public double ScaleX { get; set; } = 1;

    public double ScaleY { get; set; } = 1;

    public string GeometryJson { get; set; } = "{}";

    public string StyleJson { get; set; } = "{}";

    public int ZIndex { get; set; }

    public long Version { get; set; }
}

```

C#

В C# лучше иметь отдельные классы геометрии:

```

public sealed record LineGeometry(
    double X1,
    double Y1,
    double X2,
    double Y2);

public sealed record CircleGeometry(
    double Radius);

public sealed record RectangleGeometry(
    double Width,
    double Height,
    double RadiusX,
    double RadiusY);

```

```
public sealed record Point(double X, double Y);
```

```
public sealed record TriangleGeometry(  
    Point P1,  
    Point P2,  
    Point P3  
)
```

C#

И сериализовать соответствующую модель в `jsonb`.

Почему положение лучше отделять от геометрии

Допустим, линия имеет локальные координаты:

```
{  
  "x1": 0,  
  "y1": 0,  
  "x2": 100,  
  "y2": 100  
}
```

JSON

А в таблице:

```
x = 300  
y = 200
```

Тогда реальные координаты линии:

```
начало: 300, 200  
конец: 400, 240
```

При перемещении линии на 20 пикселей вправо:

```
update elements  
set  
  x = x + 20,
```

SQL

Геометрию вообще не нужно менять.

То же самое работает для треугольника, прямоугольника и круга.

Выделение не обязательно сохранять в PostgreSQL

Состояние:

какая фигура сейчас выделена

обычно является состоянием интерфейса, а не документа.

Его можно держать:

- в состоянии Blazor-компонента;
- в состоянии circuit;
- в браузере;
- в серверном состоянии активной сессии.

В БД его стоит сохранять только в случае, если после повторного открытия приложения пользователь должен увидеть ту же выделенную фигуру.

Если позже появится совместное редактирование, выделение других пользователей лучше передавать в реальном времени, но не обязательно постоянно записывать в PostgreSQL.

Удаление

Для редактора я бы сначала использовал мягкое удаление:

```
update elements
set
    deleted_at = now(),
```

SQL

Загрузка:

```
select *
from elements
where document_id = @documentId
```

SQL

Это упрощает:

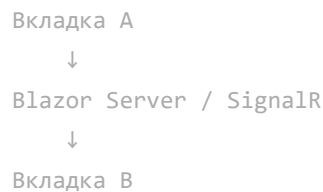
- `undo`;
- синхронизацию между вкладками;
- восстановление после ошибки;
- историю документа.

Позже старые удалённые элементы можно физически очищать.

Синхронизация двух вкладок каждые 50 мс

Очень важно: обновление второй вкладки каждые 50 мс не означает, что PostgreSQL тоже нужно обновлять каждые 50 мс.

Лучше построить поток так:



PostgreSQL здесь хранит долговременное состояние, а не является транспортом каждого движения мыши.

=

30 время движения

↑ ...

Вкладка А локально двигает фигуру без задержки.

Раз в 50 мс отправляет:

```
{
  "operation": "move",
  "documentId": "...",
  "elementId": "...",
  "x": 250,
  "y": 140,
  "sequence": 81,
```

JSON

Сервер:

1. принимает изменение;
2. обновляет состояние документа в памяти;
3. отправляет изменение всем остальным подключениям документа;

4. вкладка В применяет его;
5. более старые сообщения игнорируются по `sequence` .

Сохранение в PostgreSQL

Можно сохранять:

- раз в 200–500 мс во время перемещения;
- обязательно сразу при `pointerup` ;
- обязательно сразу при удалении;
- обязательно сразу при создании.

То есть визуальное обновление:

20 раз в секунду

А запись в PostgreSQL может происходить реже.

Это снижает:

- количество SQL-запросов;
- блокировки;
- нагрузку;
- вероятность конфликтов;
- количество записей в WAL PostgreSQL.

Версии элементов

У каждой фигуры стоит хранить:

SQL

`version bigint`

Обновление:

SQL

```
update elements
set
  x = @x,
  y = @y,
  version = version + 1,
  updated_at = now()
```

```
where id = @id
and version = @expectedVersion;
```

Если обновлено 0 строк, значит другая вкладка уже успела изменить фигуру.

Для текущего случая, где один пользователь просто открыл две вкладки, можно использовать более простое правило:

```
последнее изменение побеждает
```

Но `version` всё равно пригодится для:

- игнорирования устаревших сообщений;
- восстановления после переподключения;
- будущего совместного редактирования.

Blazor Server и несколько вкладок

Каждая вкладка Blazor Server будет иметь отдельный `circuit`. Изменение состояния в одном `circuit` автоматически не обновит другой.

Для синхронизации понадобится общий слой:

```
DocumentRealtimeService
```

который будет группировать подключения по `documentId`.

Для одного экземпляра приложения можно использовать `singleton`-сервис:

```
C#
```

```
Dictionary<Guid, DocumentRuntimeState>
```

Но обязательно с потокобезопасностью, например через:

```
C#
```

```
ConcurrentDictionary
```

или отдельные блокировки для каждого документа.

Если приложение будет запущено в нескольких экземплярах, одного `singleton` уже недостаточно. Тогда между серверами понадобится:

- Redis Pub/Sub;

- SignalR Redis backplane;
- PostgreSQL LISTEN/NOTIFY .

Но PostgreSQL LISTEN/NOTIFY лучше использовать для уведомлений, а не для сохранения каждого движения.

Как бы я строил архитектуру по этапам

Первая версия

```
documents
elements
```

В elements :

```
element_type
x
y
rotation
scale_x
scale_y
geometry jsonb
style jsonb
z_index
version
deleted_at
```

Синхронизация:

```
Blazor Server + общий realtime-сервис
```

Сохранение:

```
периодически во время drag + обязательно при завершении drag
```

Вторая версия

Добавить:

```
element_operations
```

Для:

- undo;

- redo;
- истории;
- восстановления изменений.

Третья версия

Добавить:

- snapshots документа;
- Redis для нескольких серверов;
- shared editing;
- блокировки или разрешение конфликтов;
- группы элементов;
- компоненты;
- текст;
- изображения.

Итоговая оценка вариантов

Вариант	Оценка
Весь SVG одной строкой	Хорошо для экспорта, плохо как основная модель
SVG-строка на каждый элемент	Лучше, но геометрия остаётся непрозрачной
Четыре независимые таблицы	Строго, но неудобно для общих операций
Общая таблица + дочерние таблицы	Хорошо и строго, но сложнее
Одна широкая таблица с NULL	Быстро станет неудобной
Тип + две точки bounding box	Слишком ограничено, особенно для треугольника и поворота
Тип + jsonb geometry	Лучший баланс для твоего текущего приложения
Только список операций	Мощно, но слишком сложно для первой версии
Текущее состояние + операции	Наиболее перспективная архитектура

Мой выбор для твоего проекта: одна таблица elements , обычные колонки для трансформации, jsonb для геометрии и стиля, SVG генерировать при

отображении. SVG-строку можно дополнительно сохранять как snapshot или использовать только для экспорта.

